

Transformers: Attention as a Different Inductive Bias

The capstone of the architecture-axis arc. The transformer drops the two big architectural commitments of the previous posts (the CNN’s spatial weight sharing and the RNN’s sequential recurrence) and replaces them with one new mechanism: **attention**. Every position attends to every other position directly, with the attention weights learned from data rather than baked in by the architecture. The result is fully parallelizable, has no vanishing-gradient problem, and starts from a much weaker prior than either the CNN or the RNN. Where the previous architectures committed to *which* positions matter, attention learns *which positions matter for which other positions*.

This post derives the transformer block by hand: scaled dot-product attention, multi-head attention, the causal mask, sinusoidal positional encoding, layer norm, residual connections. Every backward pass is written out. The implementation is in NumPy because pure Python loops over the $T \times T$ attention matrix are not tractable; the math is the same as everywhere else in the series, just vectorized so it finishes in minutes instead of days.

1. Where this sits in the series

We have built four architectures so far. Each commits to a specific *equivariance*, which is the algebraic name for a symmetry the architecture respects by construction:

Architecture	Weight sharing axis	Equivariance
MLP	none	input permutation (and only that)
CNN	spatial position	2-D translation in space
RNN	timestep	1-D translation in time
Bengio LM	embedding, not the head	none across the windowed positions
Transformer	across positions	permutation over positions (broken by positional encoding)

The transformer’s row in this table reads oddly because it commits to the wrong prior on purpose. Pure self-attention is permutation equivariant: shuffle the input tokens and the output tokens shuffle the same way. That is *exactly* the wrong prior for language, where order matters. The transformer fixes this by adding a **positional encoding** to the embeddings, which breaks the symmetry along the sequence axis just enough to recover order sensitivity. The model thus starts maximally flexible (no architectural commitment to which positions interact) and uses the positional encoding as a soft constraint, leaving the rest to the data.

This is the move that makes the transformer different. The CNN and RNN bake in a strong prior and learn within it. The transformer bakes in almost nothing and lets the attention weights discover the relevant structure per-input. That trades a useful prior (when it matches the data) for a much weaker prior (when it would not). The bet pays off when there is enough data to fit the attention pattern.

2. Scaled dot-product attention

Attention is a content-based lookup. Given T input vectors of dim D , stacked into a matrix $X \in \mathbb{R}^{T \times D}$, produce three views via learned linear projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

These are the **queries**, **keys**, and **values**. Each row Q_i asks “what kind of information do I want?”. Each row K_j advertises “this is what I have to offer”. Each row V_j carries the actual content to be retrieved. The attention output for position i is a weighted average of the values, with weights given by how well Q_i matches each K_j :

$$\text{Attn}(Q, K, V)_i = \sum_j a_{ij} V_j, \quad a_{i:} = \text{softmax}\left(\frac{Q_i K^\top}{\sqrt{D_h}}\right).$$

In matrix form,

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{D_h}}\right) V.$$

The $1/\sqrt{D_h}$ scale is not cosmetic. Without it, the inner products $Q_i K_j$ grow with the head dimension D_h (each is a sum of D_h products of independent random variables), and the softmax saturates for large D_h : one entry goes to 1, all others to 0, and the gradient through the softmax vanishes. Dividing by $\sqrt{D_h}$ keeps the pre-softmax scores at unit variance, which keeps the softmax in a regime where it is differentiable.

Backward through attention, by hand

Three Jacobians to derive: through the softmax, through the $QK^\top$ multiplication, and through the V averaging.

Let $A = \text{softmax}(S)$ where $S = QK^\top/\sqrt{D_h}$, and let $O = AV$. The gradient on O flows back as follows.

Through $O = AV$:

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial O} V^\top, \quad \frac{\partial L}{\partial V} = A^\top \frac{\partial L}{\partial O}.$$

Through the row-wise softmax. For each row i , $A_{ij} = e^{S_{ij}} / \sum_k e^{S_{ik}}$, and the standard derivation gives

$$\frac{\partial L}{\partial S_{ij}} = A_{ij} \left(\frac{\partial L}{\partial A_{ij}} - \sum_k A_{ik} \frac{\partial L}{\partial A_{ik}} \right).$$

In code that is one line: `dS = (dA - (dA * A).sum(axis=-1, keepdims=True)) * A`.

Through $S = QK^\top/\sqrt{D_h}$:

$$\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{D_h}} \frac{\partial L}{\partial S} K, \quad \frac{\partial L}{\partial K} = \frac{1}{\sqrt{D_h}} \left(\frac{\partial L}{\partial S} \right)^\top Q.$$

These are the only matrix derivatives needed for the attention layer. The rest is the same **Linear** backward we have been using since the walkthrough, applied separately to W_Q , W_K , W_V , and a final output projection W_O .

A subtle point: Q , K , and V are all computed from the *same* input X . The total gradient on X is the sum of the gradients that flow back through the three projection layers, because X feeds three branches in the forward computation.

3. Multi-head attention

A single attention layer with a full D -dimensional projection is restrictive: every position uses one query/key/value vector to summarize all the relations it cares about. **Multi-head attention** splits the D dimensions into H independent **heads** of dimension $D_h = D/H$, runs attention separately in each head, then concatenates the head outputs and projects back to D dimensions through W_O .

Each head can attend to a different aspect of the input: one head might track syntactic dependencies, another semantic similarity, another absolute position. The split is structural; the heads do not share parameters. With $D = 64$ and $H = 4$ we get four heads of dimension 16 each, and the total parameter count is the same as a single head with $D = 64$ (the projections collectively still map $D \rightarrow D$).

In code the split is just a reshape:

```
Q = q.reshape(T, H, D_h).transpose(1, 0, 2) # (H, T, D_h)
```

and the attention computation is batched over H . Backward unbatches with the inverse reshape.

4. Causal masking

For language modeling we predict the next token, so position i must not attend to positions $j > i$ (information from the future leaks the answer). The fix is a **causal mask**: before the softmax, set $S_{ij} = -\infty$ (in practice -10^9) for $j > i$. The softmax then puts zero weight on those entries.

```
mask = np.tril(np.ones((T, T)))
scores = np.where(mask[None] > 0, scores, -1e9)
```

The backward pass needs no special handling. Masked positions have $A_{ij} = 0$, so the softmax-backward expression above gives $\partial L / \partial S_{ij} = 0$ at masked positions too, by the $\times A$ factor.

The causal mask is what makes the model a **decoder-only** transformer (GPT-style). An encoder-only transformer (BERT-style) uses no mask, so every position attends to every other, which is appropriate for classification but unsuitable for generation.

5. Sinusoidal positional encoding

Pure attention is permutation equivariant: shuffle the input tokens and the attention outputs shuffle the same way. That is the wrong prior for sequences, where order matters. The fix is to add a position-dependent vector to each input embedding before the first attention layer:

$$X'_i = X_i + \text{PE}_i,$$

where $\text{PE}_i \in \mathbb{R}^D$ is a fixed function of the position i . Vaswani et al. used sinusoidal encodings:

$$\text{PE}_{i,2k} = \sin(i/10000^{2k/D}), \quad \text{PE}_{i,2k+1} = \cos(i/10000^{2k/D}).$$

There are no learned parameters; the positional encoding is just a constant matrix added once at the input. The geometric property that motivated this choice (each $\text{PE}_{i+\Delta}$ is a linear function of PE_i , encoding relative positions implicitly) turns out to be less important in practice than just *having* a position-dependent signal at all. Modern transformers usually use learned positional embeddings or relative-position schemes; the sinusoidal form is the simplest and is what this post uses.

This is the only place the sequence-order prior enters the architecture. Without it, the transformer would be entirely permutation equivariant over input positions.

6. Layer norm and residuals

Transformer blocks are notoriously hard to train without two stabilizers:

- **Residual connections.** Every sub-block adds its input back to its output: $y = x + f(x)$. The gradient through the addition is just the identity, so the gradient on the input has both a “through f ” path and a direct “around f ” path. This is what keeps gradients from vanishing through many stacked blocks.
- **Layer normalization.** Normalize each token vector to zero mean and unit variance along its feature dimension, then scale and shift with learned γ and β vectors. This keeps activation magnitudes from growing or shrinking through the depth of the network, which makes optimization much easier.

The backward through layer norm has a slightly involved form because μ and σ depend on x :

$$\frac{\partial L}{\partial x} = \frac{1}{\sigma} \left(\frac{\partial L}{\partial \hat{x}} - \overline{\frac{\partial L}{\partial \hat{x}}} - \hat{x} \cdot \overline{\frac{\partial L}{\partial \hat{x}}} \cdot \hat{x} \right),$$

where $\hat{x} = (x - \mu)/\sigma$ is the normalized vector and the overlines denote the mean over the feature dimension. This is the expression Ba et al. (2016) derived in the appendix; this library implements it directly.

The “pre-norm” architecture used here (**LayerNorm** *before* each sub-block, residual added at the end) is the variant that trains most reliably in deep stacks. The original “post-norm” architecture from Vaswani et al. is slightly harder to train without learning-rate warmup.

7. The full block

Each transformer block does two things in sequence, each with a residual:

$$\begin{aligned} x &\leftarrow x + \text{Attn}(\text{LN}(x)), \\ x &\leftarrow x + \text{FFN}(\text{LN}(x)). \end{aligned}$$

The **feedforward network (FFN)** is a position-wise two-layer MLP with a GELU non-linearity:

$$\text{FFN}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2,$$

with $W_1 \in \mathbb{R}^{D \times 4D}$ and $W_2 \in \mathbb{R}^{4D \times D}$. The $4\times$ expansion is a Vaswani convention and is roughly where the model’s parameter count goes. The FFN applies the same MLP independently to each position; it does no mixing across positions (that is the attention’s job).

A decoder-only transformer is then: embedding + positional encoding + N stacked blocks + a final layer norm + a linear head to vocab logits. With $D = 64$, $H = 4$, $D_{\text{ff}} = 256$, $N = 2$ blocks, and a 93-character vocabulary including three style tokens, the model has about 112,000 parameters.

8. The inductive biases of the transformer

Now we can name them precisely.

- **Permutation equivariance over positions (broken by PE).** Without positional encoding, the transformer would commute with arbitrary permutations of the input sequence. The PE breaks this exactly as much as needed to make order matter, and no more. Compare with the CNN, which commits to translation equivariance for *all* pairs of positions; or the RNN, which commits to time-translation equivariance via a recurrent state. The transformer’s prior is weaker, more flexible, and depends on the data to fill in the details.
- **Pairwise interaction at every layer.** Every output position is a weighted average over *every* input position (subject to the mask). The CNN’s local-window prior says distant positions cannot interact in one layer; the transformer says they can. This is expensive: the attention matrix is $T \times T$, so compute scales as T^2 in the sequence length. For long sequences this matters; a variety of “efficient transformer” architectures attack the T^2 cost, all by re-introducing a stronger locality prior.
- **Content-based addressing.** The attention weights a_{ij} are a function of the input via Q and K , not a function of position alone. This is the difference between “look at the previous token” (a position-based rule, hard-coded in a CNN with kernel size 2) and “look at the most recent token of the same type as me” (a content-based rule, easy in a transformer, hard everywhere else).

These add up to: the transformer has the weakest architectural prior of the four architectures in this series, and compensates with more parameters and more data. This is why transformers dominate at scale. When data is plentiful, the weaker prior is the better bet because the model can learn what the right prior is.

9. The experiment: three authors, one model

Three public-domain corpora from Project Gutenberg:

- Jane Austen, *Pride and Prejudice*
- Edgar Allan Poe, *Works* (Vol. I)
- Lewis Carroll, *Through the Looking-Glass*

Each book is stripped of the Project Gutenberg wrapper, has its front matter (title page, table of contents, publisher info) skipped, and is truncated to 120,000 characters of actual prose. The three are concatenated, each prefixed with a per-author style token: [AUSTEN], [POE], [CARROLL]. At sample time, seeding with a style token steers the model into that author’s style.

Configuration:

- Decoder-only, 2 transformer blocks

- $D = 64$, $H = 4$ heads, $D_{\text{ff}} = 256$
- Context length $T = 128$
- Sinusoidal positional encoding, pre-norm, GELU FFN
- Adam optimizer, $\beta_1 = 0.9$, $\beta_2 = 0.95$, learning rate 3×10^{-4}
- Batch size 32, 2000 training iterations

Total parameters: 112,351 (95 vocab tokens). Training takes about six minutes in NumPy on a single CPU core. The mean per-token loss descends from $\log 95 \approx 4.55$ (uniform random) to 2.32 over 2000 iterations.

Samples after training, seeded with each style token at temperature 0.9:

[AUSTEN]:

therast ke acrt, will towe thce prent,
it: be meneat. Adald ou, houly spre vooert Aly dinert Lor ad hall
and hous fitt ourk gakndiche marooth this mjue--ae cotion. alf as
I dof.

"Souend tealess Mrir."

"* she's llus liks t

[POE]:

bousthe the agingeat thoit)
he _Hurenn tor thich butt itt ach to, warly theg ictish, patich_
at llleak thecoured tlas deag." Cict fa-ges pjo po on the athe at
are brer serf ith whe the ser." Wro inge ghes dre the thousel

[CARROLL]:

ve, eand ilds ilrles,
pot wen yout. Bpors" so hen'delid," mope the ing of he the the
the gro dit the whiclett mrae terithites soust ourt muratge of
ond afong wentpad the, beres onepe gotle at, and ony she soong
the riong

These are not coherent English, and they should not be expected to be: a 112,000-parameter model with two transformer blocks, trained for 2000 iterations on 360KB of text, has nowhere near the capacity or training budget for fluent prose. What the samples *do* show is **per-author texture**, which is exactly the inductive-bias claim this experiment is set up to test:

- The [AUSTEN] sample has dialogue punctuation ("..."), sentence-final periods, the abbreviation form *Mrir*. (echoing Austen's frequent *Mr.*), and the broken-off-mid-quote that Austen's narrators do often.
- The [POE] sample has italic markup (*_word_*), parenthetical asides, dashes, and more compound-clause structure. Poe's prose in this corpus has more typographical machinery than Austen's, and the model picks it up.
- The [CARROLL] sample is more rambling, with stretches of common articles (**the the the**) and the lightly-punctuated narrative flow that *Through the Looking-Glass* uses. Less typographical ornament than Poe, less period-and-quote rhythm than Austen.

These are not literary judgments; they are the kind of low-level statistical pattern a small character-level model can pick up in six minutes of training. The textures are real but they are not prose. With more parameters and more training, the same architecture produces genuinely fluent text at each style (modern LLMs are the existence proof). The point here is not the absolute quality but that the *style signal* is recoverable from the style token alone, through nothing but the attention pattern over the conditioning context.

The full implementation is `examples/transformer.py`. The script loads the corpora, builds the tokenizer, defines all six layer types (Embedding, LayerNorm, Linear, CausalMultiHeadAttention, FFN, TransformerBlock), implements Adam, and runs the training loop. About 450 lines total, no autograd, no machine-learning framework, just NumPy.

10. Why we left scratchnn behind

The previous posts stayed in pure Python because the math was small enough to write loops for. Attention is not. A single forward pass through this network does (per layer): a 128×64 matrix multiply for each of Q, K, V; a $4 \times 128 \times 128$ attention score matrix; a $4 \times 128 \times 16$ value-weighting; an output projection; a 128×256 FFN expansion; and the inverse. Across two layers and a vocabulary projection that is on the order of 10^7 floating-point operations *per sequence*. At a NumPy matmul rate of order 10^9 multiply-adds per second per CPU core, the forward pass takes milliseconds. In pure Python with explicit loops, the same forward pass takes minutes.

The math has not changed. Every backward pass in this file was derived by hand, on paper, before being typed in. NumPy is the vectorized loop language; it is not the source of the gradients. The scratchnn walkthrough’s closing note (autograd is the natural generalization of per-layer backward) is what this post would lean on if we wanted to go further: every operation in the transformer backward is a few-line derivation in scratchnn’s style, and an autograd engine would just collect them automatically. We are not using one. The gradients here came from us.

11. Closing reflection: the inductive-bias menu

Across this series we have surveyed five architectural commitments:

1. **MLP (walkthrough)**. No structural prior. Universal approximator, data-hungry.
2. **CNN**. Translation equivariance plus locality. Right for images, wrong for non-spatial data.
3. **Bengio fixed-context LM**. A Markov prior of order N , with position-sensitive head. Right when context is small and you know how small.
4. **RNN**. Time-translation equivariance with an unbounded state. Right for sequences when long-range dependencies fit in the state.
5. **Transformer**. Almost no architectural prior; pairwise attention over all positions, with a positional encoding breaking the permutation symmetry. Right at scale, where the data can fill in the missing prior.

The output-head axis (covered in its own post) is orthogonal: identity + MSE for Gaussian targets, sigmoid + BCE for Bernoulli, softmax + CE for Categorical, Poisson for counts, and so on. Architecture and output head are independent inductive-bias axes; any architecture can pair with any matching head.

The framing the series has used throughout, and that this post restates one last time:

Every architecture is a prior. Every prior is a bet. Different bets pay off on different

data. The role of the practitioner is to know which prior matches the data on hand and to be able to derive, implement, and verify the gradient when no autograd library will do it for you.

A closing post on reinforcement learning introduces the third supervised-versus-not paradigm, but as a learning setting rather than an architecture. The architecture-axis story ends here.